ROS 2 TASK REPORT

Drone Altitude Publisher and Subscriber Nodes

Name: Tejashree H N

1. Aim

To create a ROS 2 Python publisher node that publishes simulated drone altitude values on a topic, and to understand how a subscriber node can receive those values.

2. Introduction

ROS 2 (Robot Operating System 2) is a middleware framework used to build software for robots. A node is an individual program that performs a task. Nodes can exchange data using topics. In this task, the publisher generates example altitude values and sends them through the drone/altitude topic. A subscriber can listen to the same topic and display received values. The altitude in this exercise is simulated; it is not measured from a physical drone.

3. Tools and Technologies

- Ubuntu Linux terminal
- ROS 2 and the rclpy Python client library
- Python 3
- std_msgs/msg/Float32 message type
- colcon build tool
- A ROS 2 workspace named drone_ws

4. Workspace and Package Structure

A ROS 2 workspace was used to organize the package. The package is named drone_telemetry. The file listing observed during the task showed the following structure:

```
drone_ws/
└── src/
    └── drone_telemetry/
        ├── package.xml
        ├── setup.py
        ├── setup.cfg
        ├── resource/
        │   └── drone_telemetry
        ├── drone_telemetry/
        │   ├── __init__.py
        │   └── altitude_publisher.py
        └── test/
```

The outer setup.py defines package installation metadata and the console command. The Python module directory contains the executable Python node file. The resource marker, package.xml, and setup.cfg are part of the ROS 2 Python package structure.

5. Procedure

Step 1 — Open the terminal and locate the workspace
The workspace used for this task was ~/drone_ws, with the package under ~/drone_ws/src/drone_telemetry.

Step 2 — Create or inspect the package files
The package structure was checked with the command shown below. This helps verify where the Python node and package metadata files are stored.

Step 3 — Write the publisher node
The Python file altitude_publisher.py imports rclpy, Node, and Float32. It defines a node named altitude_publisher and creates a publisher on drone/altitude.

Step 4 — Configure the executable entry point
The package setup.py includes the console-script entry point altitude_publisher = drone_telemetry.altitude_publisher:main. This tells Python packaging which main() function should run when the command is installed.

Step 5 — Build the package
The package is built from the workspace root with colcon. If the build fails, the error needs to be resolved before the console command can be expected to exist.

Step 6 — Source the workspace and run the node
After a successful build, source the ROS 2 installation and workspace setup file, then run the installed node. The exact ROS 2 distribution setup path depends on the installation.

Step 7 — Subscriber concept
A subscriber node uses create_subscription() with the same message type and topic. Its callback prints each received altitude message. The subscriber execution in this task was not confirmed because an error was encountered.

6. Important Terminal Commands
Inspect package files: `find ~/drone_ws/src/drone_telemetry -maxdepth 3 -type f`

Move to workspace root: `cd ~/drone_ws`

Build the package: `colcon build --packages-select drone_telemetry`

Source ROS 2 (replace humble if your installed distribution differs): `source /opt/ros/humble/setup.bash`

Source the built workspace: `source ~/drone_ws/install/setup.bash`

Run the publisher after a successful build: `ros2 run drone_telemetry altitude_publisher`

Check topic data while publisher is running: `ros2 topic echo /drone/altitude`

Note: Commands are included as the intended workflow. Run them in the terminal only after confirming the ROS 2 distribution and resolving any build errors. The command output should be captured from the actual system rather than recreated.

7. Publisher Code

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32


class AltitudePublisher(Node):

    def __init__(self):
        super().__init__('altitude_publisher')

        self.publisher_ = self.create_publisher(
            Float32, 'drone/altitude', 10
        )

        self.altitude = 0.0
        self.timer = self.create_timer(1.0, self.publish_altitude)

    def publish_altitude(self):
        msg = Float32()
        msg.data = self.altitude
        self.publisher_.publish(msg)

        self.get_logger().info(
            f'Drone altitude: {msg.data:.2f} meters'
        )

        self.altitude += 0.5
        if self.altitude > 10.0:
            self.altitude = 0.0


def main(args=None):
    rclpy.init(args=args)
    node = AltitudePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

8. Code Explanation

- **rclpy —** Python client library used to create and run ROS 2 nodes.
- **Node —** Base class for a ROS 2 node.
- **Float32 —** Message type for a single 32-bit floating-point value.
- **create_publisher() —** Creates a publisher for the selected message type and topic.
- **drone/altitude —** Topic name used to send altitude messages.
- **create_timer(1.0, ...) —** Calls the callback approximately once per second.
- **publish() —** Sends the message to the ROS 2 topic.
- **rclpy.spin() —** Keeps the node active so callbacks can run.

- **Queue depth 10 —** QoS history depth setting allowing a queue of up to ten messages under the selected default profile.

9. Subscriber Code

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32


class AltitudeSubscriber(Node):
    def __init__(self):
        super().__init__('altitude_subscriber')
        self.subscription = self.create_subscription(
            Float32,
            'drone/altitude',
            self.listener_callback,
            10
        )

    def listener_callback(self, msg):
        self.get_logger().info(
            f'Received drone altitude: {msg.data:.2f} meters'
        )


def main(args=None):
    rclpy.init(args=args)
    node = AltitudeSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

The subscriber uses the same topic name and message type as the publisher. When a message arrives, the callback displays the received altitude value in the terminal.

10. Issues Encountered

The package must be built and sourced correctly before running the installed node executables. If an executable is not found, check the build output, package entry point, and workspace setup file.

11. Result

The task covers the ROS 2 package structure and publisher/subscriber communication approach. The publisher generates simulated altitude values for the drone/altitude topic, and the subscriber receives and displays messages from that topic.

12. Conclusion

This task demonstrates the ROS 2 Python package structure, publisher and subscriber nodes, message types, topics, timers, callbacks, and the colcon build workflow. The publisher sends simulated altitude values and the subscriber displays the values received from the topic.

13. Short Oral Explanation

“I worked on a ROS 2 task to understand publisher and subscriber communication. I used a workspace named drone_ws and a Python package named drone_telemetry. The publisher node sends Float32 messages on the drone/altitude topic, with a simulated altitude value increasing by 0.5 meters every second. The subscriber listens to the same topic and displays each received altitude value. This task helped me understand ROS 2 nodes, topics, messages, timers, callbacks, and package execution.”